

NestJS Microservices — full testing guide

Architecture, Docker, migrations, performance at scale, and Postman chat tests

1. What we built from scratch

This monorepo is a production-style NestJS 11 platform. Clients talk only to the **API Gateway** (HTTP + Socket.IO). The gateway never opens PostgreSQL. Auth, Users, and Chat are separate microservices. Each owns its own database. RabbitMQ carries request/response between gateway and workers. Redis is used for cache, access-token blacklist, online/offline presence, JWT validate cache, conversation member cache, and the Socket.IO Redis adapter (so several gateway processes share chat rooms).

```
Client
|
v
API Gateway :3002  REST /api + Swagger /api/docs + Socket.IO /chat
|                  JWT, RBAC, validation, rate limit, inflight cap, timeouts
|
+-- RabbitMQ auth_queue --> Auth Service :3001 --> PostgreSQL AUTH_POSTGRES_DATABASE
+-- RabbitMQ user_queue --> User Service :3003 --> PostgreSQL USER_POSTGRES_DATABASE
+-- RabbitMQ chat_queue --> Chat Service :3004 --> PostgreSQL CHAT_POSTGRES_DATABASE
|
+-- Redis  cache, blacklist, presence, Socket.IO pub/sub
```

Features shipped

Area	What it does
Auth	Register, login, refresh rotation, logout blacklist, /auth/me, change password, admin bootstrap
Users	Profiles keyed by auth account id, pagination, admin list/update/deactivate
Chat REST	Private + group, members, messages with type, typing, seen, unreadCount, presence
Chat realtime	Socket.IO /chat: message, typing, seen, presence, heartbeat
Platform	Helmet, CORS, compression, Pino, Swagger, health (RabbitMQ + Redis), request ids
Data	TypeORM migrations only (synchronize is off), soft deletes, partial unique indexes

2. Docker, migrations, and run commands

Copy `.env.example` to `.env` and set secrets. Ports: Gateway **3002**, Auth **3001**, Users **3003**, Chat **3004**, Postgres **5432**, RabbitMQ **5672** / UI **15672**, Redis **6379**.

Infrastructure (Postgres + RabbitMQ + Redis)

```
npm install
sudo docker compose up -d
# or: npm run docker:up
sudo docker compose ps
sudo docker compose logs -f postgres rabbitmq redis
```

If this Postgres volume was created before chat existed, create the chat database once (names come from `.env` — example below uses `nest_ms_chat`):

```
sudo docker compose exec postgres psql -U "$POSTGRES_USER" -c 'CREATE DATABASE nest_ms_chat;'
```

```
# If the role is root: sudo docker exec -it nest-postgres psql -U root -c 'CREATE DATABASE nest_ms_chat;'
```

Migrations (schema is never auto-synced)

```
npm run migration:run          # auth + users + chat
npm run migration:auth:run
npm run migration:users:run
npm run migration:chat:run     # includes seen/type columns and scale indexes
npm run migration:auth:show
npm run migration:users:show
npm run migration:chat:show
npm run migration:chat:revert  # last chat migration only
```

Start the four Nest apps

```
npm run start:dev              # gateway + auth + users + chat (watch)
npm run start:gateway
npm run start:auth
npm run start:users
npm run start:chat
npm run build && npm run start:gateway:prod # production-style node dist/...
```

Health: <http://localhost:3002/api/health>. Swagger: <http://localhost:3002/api/docs>. RabbitMQ UI: <http://localhost:15672> (user/password from .env).

Useful checks

```
curl -s http://localhost:3002/api/health
npm test
npm run docker:down          # stop compose, keep volumes
python3 scripts/generate-chat-postman-pdf.py # regenerate this PDF
```

3. Heavy traffic — features implemented and how they work

A **single Node process cannot hold 1,000,000 open sockets**. That scale is reached by running many gateway workers behind a load balancer, many Auth/User/Chat consumers on RabbitMQ, and sized PostgreSQL/Redis. The features below are what the application does so a burst of concurrent API and chat calls is **served** instead of crashing the process or returning 503 for overflow.

3.1 FIFO in-flight queue (no 503 for extra traffic)

What: At most GATEWAY_MAX_INFLIGHT (default 2000) RabbitMQ round-trips run at the same time. Extra HTTP/WS calls are not rejected. They wait in a first-in, first-out line and run as soon as a slot frees.

How: `MicroserviceProxy.send()` calls `InflightLimiter.acquire()` before Auth/User/Chat RPC. If `active < max`, the call starts immediately. If not, the request is pushed onto a waiter list. When a call finishes, `release()` wakes the oldest waiter (the slot is inherited — active stays at max). Only when the queue is empty does active decrease. Overflow is never turned into HTTP 503.

```
Request 1..2000 -> acquire() starts RPC immediately
Request 2001+   -> acquire() waits in FIFO queue (client still connected)
RPC finishes    -> release() -> next waiter starts (no 503)
Worker hung > GATEWAY_TIMEOUT_MS -> 504 Gateway Timeout (stuck upstream, not overflow)
```

3.2 Gateway workers (Node cluster)

What: GATEWAY_WORKERS (default 1 for local). Set 0 to fork one worker per CPU core. Each worker is a full API Gateway + Socket.IO process sharing port 3002 via Node cluster.

How: `apps/api-gateway/src/main.ts` loads `.env`, then if it is the primary process and `workers > 1` it `cluster.fork()`s. A dead worker is restarted. Chat still works across workers because of the Redis adapter below. Locally leave this at 1 unless you are load-testing.

3.3 Socket.IO Redis adapter

What: Chat rooms, typing, seen, and presence events reach sockets even when users are connected to different gateway workers.

How: RedisIoAdapter creates a Redis pub client and a duplicate sub client (`maxRetriesPerRequest: null`, required for pub/sub). Socket.IO uses `@socket.io/redis-adapter`. `server.to(['conversation:id', 'user:id']).emit(...)` is broadcast through Redis so every worker delivers it. If Redis is down at boot, the gateway logs a warning and runs single-instance mode instead of crashing.

3.4 Cheap WebSocket connect + message fan-out

What: Connecting 100,000 sockets must not each run “list my 100 conversations” over RabbitMQ. Inbox messages must still arrive if the user has not opened that chat.

How: On connect the socket only joins `user:{authId}` and sets Redis presence. `chat:join` adds `conversation:{id}` when the user opens a thread. Send-message / seen RPC returns `recipientIds`. The gateway emits to the conversation room **and** every `user:{memberId}` room so the inbox updates without joining every chat on connect. `perMessageDeflate` is off (CPU), buffer max 100kb, ping 25s.

3.5 JWT VALIDATE cache

What: Every REST call used to RPC Auth Service VALIDATE. At high RPS that saturates `auth_queue`.

How: After JWT signature + blacklist checks, `AuthSessionCache` reads `auth:validate:{userId}` in Redis (TTL `AUTH_VALIDATE_CACHE_SECONDS`, default 15). Miss → VALIDATE RPC, then cache. Logout and change-password delete the key so a revoked/deactivated user is not treated as valid for the rest of the TTL. WebSocket auth uses the same cache.

3.6 Conversation member cache (typing)

What: Typing events must not call `GET_CONVERSATION` (Postgres) on every keystroke.

How: Redis SET `chat:cache:members:{conversationId}` (60s TTL) stores member account ids. Typing uses the SET when present; otherwise one `GET_CONVERSATION` fills it. Add/remove/leave members invalidates the key.

3.7 Online / offline presence in Redis

What: Presence is shared across gateway workers (not stored in process memory).

How: Connect: SADD socket id, SET online key with 45s TTL, delete `lastSeen`. Heartbeat refreshes the TTL. Last socket disconnect: DEL online, SET `lastSeen` ISO time. REST GET `/chat/presence/:userId` and conversation `members[]`. status read these keys with a Redis pipeline (one round-trip for many members).

3.8 PostgreSQL connection pool + chat indexes

What: One DB connection per request would exhaust Postgres under heavy traffic. Message history and unread counts must stay index scans, not table scans.

How: `TypeORM extra.max = POSTGRES_POOL_MAX` (default 20), min 2, idle 30s, connect timeout 5s, `statement_timeout` 10s. Extra queries wait inside the pg pool (they are not failed). Migration `1730000002000-AddChatScaleIndexes` adds (`conversationId`, `createdAt` DESC) on messages and a composite unread index on `conversation_members`. Compose sets Postgres `max_connections=200` so several service pools fit.

3.9 RabbitMQ backpressure

What: Workers must not pull more jobs than they can finish; messages must survive a restart.

How: Queues are durable; messages persistent. Workers `noAck: false` (manual ack) with `RABBITMQ_PREFETCH` default 16. The gateway `ClientProxy` keeps `noAck: true` on reply queues (manual ack there causes AMQP 406). Heartbeat 30s, reconnect 5s. Scale Auth/User/Chat processes to drain queues faster — do not put `noAck:false` on the gateway.

3.10 Timeouts, body limits, throttle, trust proxy

What: Hung workers must not hold sockets forever. Huge bodies and login floods must not take the process down. Rate limit must see the real client IP behind nginx.

How: HTTP TimeoutInterceptor (default 10s) applies only to HTTP, not WebSocket. RPC timeout is $\min(\text{GATEWAY_TIMEOUT_MS}, 8000)$ and maps to **504** if the worker never answers. JSON/urlencoded bodies capped at 64kb. Socket.IO maxHttpBufferSize 100kb. Global throttle (THROTTLE_LIMIT / THROTTLE_TTL_SECONDS); login/register 8/min; refresh 20/min. Health is @SkipThrottle so probes never 429. Express trust proxy so throttle uses X-Forwarded-For. Helmet + compression on HTTP; WS compression off.

3.11 Redis, logs, Docker heap

How: Gateway Redis: reconnect backoff, enableOfflineQueue false on the main client (fail fast, do not buffer millions of commands). Adapter pub/sub keeps offline queue on. Redis Compose uses volatile-lru so only TTL keys (presence, cache, blacklist) may be evicted. Production Pino autoLogging: false (access logs would dominate disk at high RPS). Service Dockerfiles set NODE_OPTIONS=-max-old-space-size=512.

3.12 Env cheat sheet

Variable	Default	Role under heavy traffic
GATEWAY_MAX_INFLIGHT	2000	Max concurrent RPCs; extras wait FIFO (no 503)
GATEWAY_TIMEOUT_MS	10000	HTTP timeout; RPC cap 8s then 504
GATEWAY_WORKERS	1	Cluster size; 0 = one worker per CPU
AUTH_VALIDATE_CACHE_SECONDS	15	Skip Auth VALIDATE RPC on cache hit
POSTGRES_POOL_MAX / MIN	20 / 2	Shared DB connections per service
RABBITMQ_PREFETCH	16	Unacked jobs per worker
THROTTLE_LIMIT / TTL	60 / 60s	Per-IP HTTP rate limit
CACHE_TTL_SECONDS	60	Gateway HTTP cache TTL

Local soak test: keep GATEWAY_WORKERS=1. Raise POOL_MAX only if Postgres max_connections has room (Compose allows 200). In production: reverse proxy, several gateway replicas, extra chat-service consumers so chat_queue does not grow. Apply indexes with `npm run migration:chat:run`.

4. Postman setup

Create an environment (for example **Nest Chat Local**):

Variable	Example	Used for
baseUrl	http://localhost:3002/api	All REST requests
accessTokenA	(from login A)	User A Authorization header
accessTokenB	(from login B)	User B Authorization header
userIdA	(from /auth/me)	Account id of user A
userIdB	(from /auth/me)	Account id of user B
conversationId	(from create private)	Chat id for messages / seen / typing
messageId	(from send message)	Optional cursor for mark-seen

On every request: **Authorization** → **Bearer Token** = `{{accessTokenA}}` or `{{accessTokenB}}`. Do not send a user profile id; chat **userId** is the auth account id from GET /auth/me (**data.id**).

5. Login as two users

You need two accounts so you can test typing, seen, and presence. Register if needed, then log in twice (two Postman tabs or switch tokens).

Step	Method	Path	Body
1	POST	{{baseUrl}}/auth/register	{"email":"a@test.com","password":"Passw0rd!","firstName":"Ada","lastName":"A"}
2	POST	{{baseUrl}}/auth/register	{"email":"b@test.com","password":"Passw0rd!","firstName":"Bob","lastName":"B"}
3	POST	{{baseUrl}}/auth/login	{"email":"a@test.com","password":"Passw0rd!"}
4	POST	{{baseUrl}}/auth/login	{"email":"b@test.com","password":"Passw0rd!"}
5	GET	{{baseUrl}}/auth/me	Bearer token A or B

From each login response save **data.accessToken**. From each **/auth/me** save **data.id** as **userIdA** / **userIdB**.

Other HTTP routes (all JWT unless Public)

Method	Path	Notes
POST	{{baseUrl}}/auth/logout	Blacklists access token; optional refreshToken body
POST	{{baseUrl}}/auth/refresh	Rotate tokens (public, tighter throttle)
PATCH	{{baseUrl}}/auth/password	Change password; current token is revoked
GET	{{baseUrl}}/users/me	My profile (userId = auth id)
GET	{{baseUrl}}/users	Admin: paginated profiles
GET	{{baseUrl}}/health	Public; RabbitMQ + Redis; not rate-limited
POST	{{baseUrl}}/chat/conversations/:id/members	Group: add members (owner/admin)
DELETE	{{baseUrl}}/chat/conversations/:id/members/:userId	Group: remove member
POST	{{baseUrl}}/chat/conversations/:id/leave	Leave group
PATCH	{{baseUrl}}/chat/conversations/:id	Rename group

6. REST: conversations and messages (type)

Every message has a **type** field. The supported value today is text. Omit it and the server defaults to text.

Method	Path	Body / notes
POST	{{baseUrl}}/chat/private	As user A: {"userId":"{{userIdB}}". Save data.id as conversationId.
POST	{{baseUrl}}/chat/groups	{"name":"Weekend trip","memberIds":["{{userIdB}}"]}
GET	{{baseUrl}}/chat/conversations	Check unreadCount and members[].status (online/offline).
GET	{{baseUrl}}/chat/conversations/{{conversationId}}	Membership required. Members include lastReadAt and status.
POST	{{baseUrl}}/chat/conversations/{{conversationId}}/messages	{"body":"Hello from A","type":"text"}. Save data.id as messageId.
GET	{{baseUrl}}/chat/conversations/{{conversationId}}/messages	Newest first. Each item has type and seenBy (array of user ids).

Expected send-message payload

```
{
```

```

    "success": true,
    "message": "Message sent successfully",
    "data": {
      "id": "<message uuid>",
      "conversationId": "<conversation uuid>",
      "senderId": "<user A id>",
      "body": "Hello from A",
      "type": "text",
      "seenBy": [],
      "createdAt": "2026-08-25T19:00:00.000Z"
    }
  }
}

```

7. Typing

Typing is a live event. REST is enough to exercise it from Postman; connected Socket.IO clients in that conversation receive **chat:typing**.

```

POST {{baseUrl}}/chat/conversations/{{conversationId}}/typing
Authorization: Bearer {{accessTokenA}}
Body: { "typing": true }

```

Then send { "typing": false } when A stops typing.

To observe the event: open a Socket.IO request as user B (section 10) and listen for **chat:typing**. The payload is { conversationId, userId, typing }.

8. Seen receipts

Each member has a **lastReadAt** cursor. When B marks the chat as seen, messages whose **createdAt** is on or before that cursor list B in **seenBy** (the sender is never included).

```

# As user B, after A sent a message:
POST {{baseUrl}}/chat/conversations/{{conversationId}}/seen
Authorization: Bearer {{accessTokenB}}
Body (mark everything now): {}
or Body (up to one message): { "messageId": "{{messageId}}" }

```

```

Then as user A:
GET {{baseUrl}}/chat/conversations/{{conversationId}}/messages
# data.items[n].seenBy should include userIdB
GET {{baseUrl}}/chat/conversations
# unreadCount for that chat should drop to 0 for user B

```

Socket.IO equivalent: emit **chat:seen** with { conversationId, messageId? }. The server broadcasts **chat:seen** to the room.

9. Online / offline presence

A user is **online** while they hold at least one authenticated Socket.IO connection on namespace **/chat**. Disconnect (or token expiry) sets them **offline** and stores **lastSeenAt** in Redis. REST presence does not require a socket — it only reads that Redis state.

```

GET {{baseUrl}}/chat/presence/{{userIdB}}
Authorization: Bearer {{accessTokenA}}

```

```

Offline example:
{ "data": { "userId": "<B>", "status": "offline", "lastSeenAt": null } }

```

```

After B connects Socket.IO /chat:
{ "data": { "userId": "<B>", "status": "online", "lastSeenAt": null } }

```

```

After B disconnects:
{ "data": { "userId": "<B>", "status": "offline", "lastSeenAt": "<ISO time>" } }

```

Conversation list and get-conversation also overlay live status on `members[].status` and `members[].lastSeenAt`. Clients should listen for `chat:presence` so the UI updates without polling.

10. Socket.IO in Postman

Postman supports Socket.IO (not the generic WebSocket request type). New → Socket.IO. Use two requests (user A and user B) if you want to watch events from both sides.

- **Server URL:** `http://localhost:3002` (no /api prefix)
- **Namespace:** `/chat`
- **Handshake → Auth:** JSON `{"token":"{{accessTokenB}}"}` You can also send header `Authorization: Bearer {{accessTokenB}}`.
- **Connect.** The socket joins only user : `{yourId}` (no list-all-chats query).
- **Emit `chat:join`** with `{"conversationId":"{{conversationId}}"}` so typing/presence use the conversation room. Inbox messages still arrive on the user room.
- **Listen for events:** `chat:message`, `chat:typing`, `chat:seen`, `chat:presence`.
- **Keep the connection alive** with emit `chat:heartbeat` every ~20 seconds (refreshes the online TTL).

Client → server events

Event	Payload	What it does
<code>chat:join</code>	<code>{"conversationId":""}</code>	Join a conversation room
<code>chat:message</code>	<code>{"conversationId":"","body":"Hi","type":"text"}</code>	Send a message; room gets <code>chat:message</code>
<code>chat:typing</code>	<code>{"conversationId":"","typing":true}</code>	Broadcast typing true/false
<code>chat:seen</code>	<code>{"conversationId":"","messageId":""}</code>	Mark seen; room gets <code>chat:seen</code>
<code>chat:heartbeat</code>	(empty)	Stay online (TTL refresh)

Server → client events

Event	Payload shape
<code>chat:message</code>	Message object (id, body, type, seenBy, senderId, ...)
<code>chat:typing</code>	<code>{ conversationId, userId, typing }</code>
<code>chat:seen</code>	<code>{ conversationId, userId, lastReadAt, messageId }</code>
<code>chat:presence</code>	<code>{ conversationId, userId, status: online offline, lastSeenAt }</code>

11. Suggested Postman sequence

- Login A and B. Save tokens and user ids.
- As A: POST `/chat/private` with B's userId. Save conversationId.
- GET `/chat/presence/{{userIdB}}` → expect **offline**.
- Connect Socket.IO as B with B's token.
- GET `/chat/presence/{{userIdB}}` → expect **online**. GET conversations as A and confirm `members[].status`.
- As A REST: POST typing true, then send a text message. B's Socket.IO tab should show `chat:typing` then `chat:message`.
- As B REST or Socket.IO: mark seen. As A, GET messages and confirm `seenBy` includes B.

- Disconnect B's Socket.IO. GET presence → **offline** with lastSeenAt set. A should receive chat:presence.

12. Common mistakes

- Using a user **profile** id instead of the auth account id from /auth/me.
- Pointing Socket.IO at `http://localhost:3002/api/chat` — the namespace is /chat on the gateway host, not under the REST prefix.
- Creating a raw WebSocket request in Postman instead of a **Socket.IO** request.
- Expecting REST GET /chat/presence to flip to online without a Socket.IO connection.
- Forgetting `npm run migration:chat:run` (seen/type columns and scale indexes).
- Calling chat on auth :3001 or users :3003 — only the gateway (:3002) exposes HTTP and Socket.IO.

13. Quick reference

Feature	REST	Socket.IO
Message type	POST .../messages { "type": "text" }	chat:message type: text
Typing	POST .../typing { typing }	emit/listen chat:typing
Seen	POST .../seen { messageId? }	emit/listen chat:seen
Online / offline	GET /chat/presence/:userId	listen chat:presence; connect = online

Swagger UI at `http://localhost:3002/api/docs` also covers these REST routes. Authorize there with the same access token if you prefer the browser over Postman.